Laboratorio de Git — Flujo colaborativo en GitHub

Propósito del laboratorio

En este laboratorio vais a trabajar con un flujo colaborativo en GitHub parecido al que necesitaréis en el proyecto final del BootCamp. El objetivo no es solo subir cambios, sino aprender a organizar el trabajo de equipo con una dinámica razonable:

- repositorio compartido;
- una rama por tarea;
- commits con intención;
- Pull Requests;
- revisión cruzada;
- rechazo y corrección de una PR;
- resolución de un conflicto controlado;
- integración ordenada en `main`.

Este laboratorio debe ayudarte a entender cómo debería trabajar un equipo pequeño sobre un mismo repositorio sin mezclar tareas ni integrar cambios a ciegas.

Modalidad

Este laboratorio está pensado para grupos de **3 o 4 personas**.

La intención no es que una sola persona lleve el control de GitHub mientras el resto espera, sino que todo el grupo entienda el flujo completo.

Forma de trabajo

Aunque cada fase tenga una persona responsable principal, **todo el grupo debe ver qué está haciendo esa persona en cada paso**.

La idea del laboratorio no es repartirse tareas para “terminar antes”, sino que todas las personas del grupo vean y entiendan:

- cómo se crea el repositorio;
- cómo se invita a colaboradores;
- cómo se crea una rama;

- cómo se abre una Pull Request;
- cómo se revisa;
- cómo se corrige;
- cómo se resuelve un conflicto;
- cómo se integra en `main`.

Objetivos

Al terminar este laboratorio deberíais ser capaces de:

- crear y configurar un repositorio compartido en GitHub;
- invitar al resto del equipo como colaboradores;
- repartir tareas simples y reconocibles;
- trabajar con una rama por tarea;
- escribir commits con Conventional Commits;
- publicar ramas en remoto;
- abrir Pull Requests claras;
- revisar el trabajo de otra persona;
- corregir una PR rechazada;
- resolver un conflicto de merge controlado;
- integrar trabajo en `main` con orden.

Tamaño y organización del grupo

Tamaño recomendado

Tamaño del grupo	Recomendación
3 personas	Válido
4 personas	Ideal
Más de 4	No recomendado

Roles orientativos

Rol	Función principal
Persona A	coordina el arranque del repositorio
Persona B	desarrolla una tarea documental
Persona C	desarrolla otra tarea documental
Persona D	desarrolla una tercera tarea y participa activamente en revisión

Si sois 3 personas

La coordinación inicial puede recaer en una de las personas que también tendrá tarea de contenido. Lo importante es que todas las personas del grupo trabajen con ramas, revisión e integración.

Importante

Aunque haya una persona que ejecute cada paso, el resto del grupo no debe desconectarse del proceso. Todo el grupo debe seguir lo que se está haciendo y entender por qué se hace así.

Escenario del laboratorio

Vais a crear un pequeño repositorio documental sobre buenas prácticas de trabajo con Git y GitHub. No se busca complejidad técnica en el contenido, sino practicar un flujo de colaboración realista y útil para el proyecto final.

El laboratorio se centrará en:

- repartir trabajo;
- aislar tareas en ramas;
- publicar ramas;
- revisar antes de integrar;
- corregir cuando haga falta;
- provocar y resolver un conflicto controlado;
- cerrar el trabajo en `main` de forma ordenada.

Estructura del proyecto

Cread una carpeta llamada `laboratorio-colaborativo-github`.

Dentro de ella, preparad esta estructura:

```
laboratorio-colaborativo-github/
├─ README.md
├─ normas-equipo.md
├─ docs/
│  ├─ ramas.md
│  ├─ pull-requests.md
│  ├─ revisiones.md
│  └─ buenas-practicas.md
└─ .gitignore
```

Contenido inicial sugerido

Archivo `README.md`

```
# Laboratorio colaborativo en GitHub

Este repositorio se utiliza para practicar un flujo de trabajo en equipo con
Git y GitHub.

## Objetivo

Aprender a organizar ramas, commits, Pull Requests, revisiones e integraciones
de forma ordenada.
```

Archivo `normas-equipo.md`

```
# Normas del equipo

- No trabajar directamente sobre `main`
- Una rama por tarea
- Commits con mensajes claros
- Revisar antes de integrar
- Corregir una PR si recibe observaciones
```

Archivo `docs/ramas.md`

```
# Ramas

Las ramas permiten aislar trabajo y evitar mezclar tareas distintas en la rama principal.
```

Archivo `docs/pull-requests.md`

```
# Pull Requests

Una Pull Request es una propuesta de integración que permite revisar una rama antes de incorporarla a otra.
```

Archivo `docs/revisiones.md`

```
# Revisiones

Revisar cambios ayuda a detectar errores, mejorar claridad y validar que una rama está bien acotada.
```

Archivo `docs/buenas-practicas.md`

```
# Buenas prácticas

Trabaja con ramas cortas, revisa antes de integrar y evita commits ambiguos.
```

Archivo `.gitignore`

```
*.log
temp/
.DS_Store
.vscode/
```

Convenciones obligatorias

Convención de ramas

Cada tarea debe vivir en una rama con un nombre claro.

Tipo de tarea	Ejemplo
Documentación	`docs/mejora-ramas`
Documentación	`docs/mejora-pull-requests`
Documentación	`docs/mejora-revisiones`
Documentación	`docs/mejora-buenas-practicas`

Convención de commits

Usad, como mínimo, estos prefijos:

- `docs:`
- `fix:`
- `chore:`

> **Regla del laboratorio**
>
> Nadie debe trabajar directamente sobre `main`.

Flujo del laboratorio

Fase 1. Preparación del repositorio compartido

Responsable principal

Persona A.

Qué debe hacer

1. Crear la carpeta del proyecto.
2. Añadir la estructura y el contenido inicial.
3. Inicializar Git.
4. Realizar un commit inicial.
5. Crear el repositorio en GitHub, marcándolo como **privado**.
6. Conectar el repositorio local con el remoto.
7. Publicar la rama principal.

8. Invitar al resto del grupo como colaboradores en GitHub.

Qué debe hacer el resto del grupo

Mientras la Persona A ejecuta estos pasos, el resto del grupo debe observar el proceso y comprobar que entiende:

- qué se está haciendo;
- por qué el repositorio debe ser privado;
- cómo se conecta el repositorio local con GitHub;
- qué significa publicar `main`;
- cómo se añaden colaboradores.

Comprobaciones obligatorias

Antes de seguir, comprobad:

- que el repositorio existe en GitHub;
- que es **privado**;
- que todo el grupo tiene acceso;
- que `main` está publicada;
- que el contenido base aparece correctamente.

> ⓘ **Punto de control**
>
> Antes de seguir, el grupo debe poder responder:
>
> - ¿qué diferencia hay entre el repositorio local y el remoto?
> - ¿por qué no vamos a trabajar directamente sobre `main`?

Fase 2. Reparto de tareas

Tareas recomendadas

Repartid estas tareas entre las personas del grupo:

Persona	Archivo asignado	Objetivo mínimo
Persona A	`docs/ramas.md`	ampliar la explicación sobre por qué usar ramas
Persona B	`docs/pull-requests.md`	explicar qué debe incluir una PR útil
Persona C	`docs/revisiones.md`	añadir criterios básicos para revisar una rama
Persona D	`docs/buenas-practicas.md` o `README.md`	ampliar recomendaciones de trabajo en equipo

Regla de trabajo

Cada persona trabajará solo en su tarea durante esta fase.

Qué debe hacer el grupo

Todo el grupo debe tener claro:

- qué tarea tiene cada persona;
- por qué cada tarea vivirá en una rama propia;
- qué archivo va a tocar cada integrante.

> ⚠ **Importante**
>
> No empecéis a editar sobre `main`. Primero cada persona debe actualizar su base local, crear su rama y cambiarse a ella.

Fase 3. Trabajo individual en rama

Responsable principal

Cada persona sobre su propia tarea.

Tarea de cada persona

1. Actualizar su copia local de `main`.
2. Crear su rama de tarea.

3. Cambiarse a esa rama.
4. Mejorar el archivo asignado.
5. Hacer uno o dos commits con intención.
6. Publicar la rama en GitHub.

Requisitos de contenido

Cada mejora debe ser suficiente como para que pueda revisarse con sentido. No basta con cambiar una sola palabra.

Requisitos de commits

- usar Conventional Commits;
- evitar mensajes ambiguos;
- no mezclar varias tareas en la misma rama.

Qué debe hacer el resto del grupo

Mientras cada persona realiza su parte, el resto del grupo debe ver y entender:

- cómo se crea una rama;
- en qué momento se hace commit;
- cuándo se publica la rama;
- qué se está subiendo realmente a GitHub.

Comprobaciones individuales

Cada persona debe revisar:

- que no está en `main`;
- que su rama tiene un nombre claro;
- que el alcance del cambio está bien delimitado;
- que su rama ya existe en GitHub.

> **Criterio profesional**
>
> Una rama buena no es la que “tiene cambios”, sino la que contiene una tarea clara, concreta y revisable.

Fase 4. Pull Request

Responsable principal

Cada persona sobre su propia rama.

Tarea de cada persona

Cada integrante debe abrir una Pull Request real en GitHub o, si el entorno no lo permite, redactar una PR simulada completa.

La PR debe incluir

- rama de origen;
- rama de destino;
- resumen del cambio;
- lista breve de cambios principales;
- qué conviene revisar.

Plantilla sugerida

```
## Resumen

Esta rama mejora la documentación sobre ramas de trabajo.

## Cambios principales

- amplía la explicación sobre ramas
- añade una recomendación sobre nombres de rama
- aclara por qué no conviene trabajar sobre `main`

## Revisión solicitada

Revisar si la explicación es clara, si la rama está bien acotada y si falta
algún ejemplo útil.
```

Qué debe hacer el resto del grupo

Cuando una persona abra su PR, el resto debe ver:

- qué rama sale;
- a qué rama va;
- cómo se describe el cambio;
- por qué no conviene integrar una rama sin una PR clara.

Comprobación

Antes de pasar a revisión, aseguraos de que la PR permite responder rápidamente a estas preguntas:

- ¿qué resuelve?
- ¿qué ha cambiado?
- ¿qué debería revisar otra persona?

> **Regla del laboratorio**
>
> No se integra ninguna rama sin PR y sin revisión previa.

Fase 5. Revisión cruzada

Responsable principal

Cada persona revisará la PR de otra persona del grupo.

Aspectos mínimos a revisar

Aspecto	Pregunta
Objetivo	¿Se entiende qué cambia?
Alcance	¿La rama está bien acotada?
Commits	¿Los mensajes son claros?
Calidad	¿El contenido añadido tiene sentido?
Limpieza	¿Hay algo que sobra o algo que falta?

Resultado obligatorio

Toda revisión debe terminar en una de estas dos opciones:

- **Aprobada**
- **Necesita cambios**

Ejemplos de comentarios útiles

- “La explicación está bien, pero falta un ejemplo concreto.”
- “La rama está bien delimitada, pero el commit podría tener un mensaje más preciso.”
- “Has mezclado una mejora de contenido con un ajuste del README; convendría separarlo.”
- “La PR se entiende, pero la descripción es demasiado breve.”

Qué debe hacer el grupo

Cuando una persona revise, el resto debe ver:

- qué se está revisando;
- qué clase de comentario tiene sentido;
- qué diferencia hay entre aprobar y pedir cambios;
- por qué revisar no es simplemente “mirar por encima”.

> **Criterio de revisión**
>
> Revisar no es buscar defectos por inercia. Revisar es comprobar si la rama está lista para formar parte de `main`.

Fase 6. PR rechazada y corrección obligatoria

Regla del laboratorio

Al menos **una Pull Request del grupo debe ser rechazada inicialmente** y corregida después.

Finalidad pedagógica

Esto no se hace para “poner una trampa”, sino para que el grupo practique un caso muy real:

- una rama no está lista;
- recibe feedback;
- se corrige;
- se vuelve a revisar;
- entonces sí puede integrarse.

Responsable principal

La persona cuya PR ha sido rechazada.

Tarea de esa persona

1. leer la observación recibida;
2. corregir el contenido o la descripción de la PR si hace falta;
3. hacer el commit adicional necesario;
4. volver a publicar la rama;
5. actualizar la PR;
6. solicitar nueva revisión.

Qué debe hacer el grupo

Todo el grupo debe ver:

- qué motivó el rechazo;
- cómo se corrige una PR;
- cómo se actualiza la rama;
- qué cambia en la segunda revisión.

Comprobación

El grupo debe comprobar que:

- la PR rechazada ha sido realmente corregida;
- la segunda revisión tiene sentido;
- la integración posterior ya está justificada.

> ✓ **Objetivo importante**
>
> Si una PR ha sido rechazada, corregida y luego aprobada, el grupo ya ha vivido una parte esencial del trabajo real con GitHub.

Fase 7. Conflicto controlado

Objetivo

Ahora vais a provocar un conflicto de merge de forma intencional dentro del flujo colaborativo.

Responsable principal

Dos personas del grupo, con el resto observando todo el proceso.

Escenario obligatorio

Dos personas del grupo deben modificar la misma línea del archivo `README.md` en ramas distintas.

Línea base a modificar

Añadid primero en `README.md` este bloque en `main` si no existe ya:

```
## Norma principal

Trabaja en ramas de tarea y revisa antes de integrar.
```

Modificación de la primera rama

Una persona debe cambiar esa línea por:

```
Trabaja en ramas de tarea, revisa antes de integrar y sincroniza con
frecuencia.
```

Modificación de la segunda rama

Otra persona debe cambiar esa misma línea por:

```
Trabaja en ramas cortas, evita mezclar cambios y no uses `pull` sin revisar
antes.
```

Flujo esperado

1. Se integra una de las ramas primero.
2. Después se intenta integrar la segunda.
3. Debe aparecer un conflicto.
4. El grupo lo resuelve.
5. Se completa la integración.

Qué debe hacer el grupo

- identificar el archivo en conflicto;
- leer los marcadores;
- decidir una resolución coherente;
- dejar el archivo limpio;

- completar la integración.

Qué debe observar todo el grupo

Todo el grupo debe ver:

- cómo aparece el conflicto;
- qué archivo queda afectado;
- cómo se leen los marcadores;
- cómo se decide la versión final.

> ⚠ **Importante**
>
> No deis por terminado el conflicto cuando desaparezcan las marcas. Solo está terminado cuando el texto final tiene sentido y el repositorio vuelve a estar en buen estado.

Fase 8. Integración ordenada en `main`

Responsable principal

La persona o personas que integren las ramas, con supervisión del resto del grupo.

Tarea del grupo

Cuando una rama esté revisada y aprobada, integradla en `main`.

Reglas de integración

- integrad una rama cada vez;
- comprobad el estado de `main` antes de cada integración;
- aseguraos de que todo el grupo sabe qué rama se va a integrar;
- revisad el historial después de cada integración.

Qué debe hacer el grupo

Todo el grupo debe ver y entender:

- qué rama se va a integrar;
- si está realmente aprobada;
- qué cambia en `main`;

- cómo queda el historial tras la integración.

Qué debéis comprobar

- que el contenido aparece correctamente en `main`;
- que el historial sigue siendo razonablemente legible;
- que no quedan ramas “a medias” sin revisar.

> **Regla del laboratorio**
>
> No integréis por prisa. Si una rama no está clara, revisadla de nuevo antes de incorporarla a `main`.

Fase 9. Verificación final

Tarea final del grupo

Cuando todas las ramas obligatorias estén integradas, comprobad:

- que `main` contiene las mejoras esperadas;
- que la PR rechazada ha sido corregida y finalmente aprobada;
- que el conflicto controlado se ha resuelto correctamente;
- que el repositorio final es coherente.

Tabla de comprobación final

Comprobación	Sí / No
Hemos creado el repositorio compartido	
El repositorio es privado	
Hemos publicado `main` en GitHub	
Todo el grupo tiene acceso al repositorio	
Cada persona ha trabajado en una rama propia	
Cada rama tiene un nombre claro	
Los commits usan mensajes comprensibles	

Comprobación	Sí / No
Cada persona ha abierto o redactado su PR	
Cada PR ha pasado por revisión	
Al menos una PR ha sido rechazada y corregida	
Hemos provocado y resuelto un conflicto controlado	
Hemos integrado el trabajo en `main` con orden	

Entregable esperado

Debéis conservar como evidencia:

- el repositorio compartido del grupo en GitHub;
- las ramas publicadas;
- las Pull Requests;
- el historial final de `main`;
- el archivo `reflexion-colaborativa.md`.

Archivo de reflexión del grupo

Cread un archivo llamado `reflexion-colaborativa.md` y responded brevemente:

1. ¿Qué diferencia habéis notado entre trabajar solos y trabajar en un repositorio compartido?
2. ¿Por qué no conviene trabajar directamente sobre `main`?
3. ¿Qué ha aportado la revisión cruzada?
4. ¿Qué habéis aprendido de la PR rechazada y corregida?
5. ¿Qué os ha aportado ver todo el proceso aunque cada paso lo ejecutara una persona distinta?
6. ¿Qué os ha parecido más delicado al resolver el conflicto?
7. ¿Qué norma de trabajo os parece más importante para el proyecto final del BootCamp?

Cierre reflexivo

Este laboratorio no busca que “sepáis usar GitHub” de forma superficial. Busca que empecéis a trabajar como un equipo pequeño con un flujo cercano al que deberíais aplicar en el proyecto final.

Qué se considera éxito en este laboratorio

Nivel	Indicador
Básico	El grupo crea el repo, trabaja en ramas y publica PRs
Medio	El grupo revisa, corrige e integra con orden
Alto	El grupo mantiene claridad en el flujo, resuelve un conflicto controlado y comprende el proceso completo aunque los pasos se repartan entre distintas personas

Recursos de apoyo

- Git Book en español: `https://git-scm.com/book/es/v2`
- Documentación oficial de Git: `https://git-scm.com/docs/git/es`
- Documentación de GitHub: `https://docs.github.com/es`
- Learn Git Branching: `https://learngitbranching.js.org/?locale=es_ES`
- Explain Git with D3: `https://onlywei.github.io/explain-git-with-d3/`
- Git Exercises: `https://gitexercises.fracz.com/`